

Advanced C Course

Code Quality applied to C - Session 3

Pascal Rothnemer

16 February 2026, EPITA

Course Program - Sessions 1-7

1) Hello, World !

- C/C++ presentation and history
- Code quality : Compliance with Specs, Readability
- IDE setup

2) Types, objects, basic pointers

- Developing on VS 2022
- Code quality : Useability, Documentation

3) C++, OOP, Classes

- Classes, OOP
- Code quality : Useability, Documentation

4) Functions, libraries

5) Arrays, Vectors

- Debugging on VS 2022
- Code quality : Robustness, exception handling

6) Loops

- metrics on VS 2022
- Code quality : Performance, metrics , optimization

7) Projects Presentations & Evaluation

Session 3

«The role of a good software is to make the complex appear to be simple »

Session 3

Technical debt illustration:-)



Session 3

«C is a low-level language with high-level elements. »

(The C Programming Language)

Session 3 – Structured Programming

“Divide each difficulty into as many parts as is feasible and necessary to resolve it.”

René Descartes (1596-1650), « Discourse on the Method »

Session 3 – Content

1) Course

- C features - contd
- C++ - Object Oriented Programming

2) Hands-on on exercises – projects definition

Keywords - Structure

Struct - defines a **composite, new, custom data type** - occupying a block of memory :

```
struct Point2D {
```

```
    float x;    // abscissa
```

```
    float y;    // ordinate
```

```
};
```

```
struct Point2D P = { 1.5, -2.0 };
```


Structures

Can contain various data types including structures.

```
struct point2D {
```

```
    float x; // abscissa
```

```
    float y; // ordinate
```

```
};
```

```
struct point3D {
```

```
    point2D p2D;
```

```
    float z; // z-axis coordinate
```

```
};
```

Structures

```
point2D My2DPoint;
```

```
My2DPoint.x = 2.0;
```

```
My2DPoint.y = -1.5;
```

```
point3D My3DPoint;
```

```
My3DPoint.p2d = My2DPoint;
```

```
My3DPoint.z = 0;
```

Accessing a Structure : directly or by a pointer

Direct Access

```
point2D My2DPoint;  
My2DPoint.x = 2.0;  
My2DPoint.y = -1.5;
```

Indirect Access

```
point2D* pPt; // pointer to  
structure point2D  
pPt = &My2DPoint;  
pPt->x = 3.3;  
pPt->y = -2.2;
```

Keywords - Union

Union - like struct, except that each data member is located at the same memory address.

Primary use : allowing access to a common location by different data types

```
union {
```

```
    float f;
```

```
    uint32_t d; // Assumes float is 32 bits wide
```

```
};
```

```
f = 3.14f;
```

```
std::cout << "Hexadecimal representation of 3.14f:" << std::hex << d << '\n';
```

Unions

C unions are used to save memory. To better understand an union, think of it as a chunk of memory that is used to store variables of different types. When we want to assign a new value to a field, then the existing data is replaced with new data.

DON'T USE UNION IF NOT TO SAVE MEMORY (TYPICALLY EMBEDDED SOFTWARE WITH LIMITED RESOURCES)

Typedef keyword

Creates an **alias** for a **data type** - does **not** create a new type

```
typedef float Temperature;
```

```
Temperature today = 25.5;
```

```
Temperature tomorrow = 18.6;
```

Typedef keyword

simplifies declaring structures :

// Without typedef:

```
struct Car {  
    char brand[30];  
    int year;  
};  
  
// declaration needs "struct"  
struct Car car1 = { "BMW", 1999 };
```

- // With typedef:
typedef struct {
 char brand[30];
 int year;
} Car;

// shorter with typedef
Car car2 = { "Ford", 1969 };

Enum keyword

- An **enum** (enumeration) type is a **data type** made of **integral constants**.

```
enum Level { LOW, MEDIUM, HIGH };
```

```
// by default first value is 0 and then increase by one
```

```
// LOW = 0, MEDIUM = 1, HIGH = 2
```


Enum keyword

- better written (readability) :

```
enum Level {  
    LOW,  
    MEDIUM,  
    HIGH  
};
```

Enum keyword

// One may force values of enum elements during declaration

```
enum Level {  
    LOW = 0,  
    MEDIUM = 10,  
    HIGH = 100 ;  
};
```

Enum keyword

// New Type « Level » Declaration. Blueprint created

```
enum Level{LOW=0, MEDIUM = 10, HIGH = 100};
```

// To create a variable of this type :

```
Level mylevel = MEDIUM; // declaring a new variable of type Level
```

Exercise – what is the output ?

```
#include <iostream>

using namespace std;

int main(){

enum color_type { red, orange, yellow, green, blue, violet };

color_type shirt, pants;

shirt = red;

pants = blue;

cout << shirt << " " << pants << endl;

Return 0;

}
```

Exercise – what is the output ?

```
#include <iostream>

using namespace std;

int main(){

enum color_type { red, orange, yellow, green, blue, violet };

color_type shirt, pants;

shirt = red;

pants = blue;

cout << shirt << " " << pants << endl;

Return 0;

}
```

0 4

Exercise – what is the output ?

```
#include <iostream>

using namespace std;

int main(){

enum weekdays {Mon, Tue, Wed, Thur, Fri, Sat, Sun};

enum weekdays day;

day = Wed;

cout << day;

Return 0;

}
```

Exercise – what is the output ?

```
#include <iostream>

using namespace std;

int main(){

enum weekdays {Mon, Tue, Wed, Thur, Fri, Sat, Sun};

enum weekdays day;

day = Wed;

cout << day;

Return 0;

}
```

Output : 2

Exercise – what is the output ?

```
int main(){  
    int n = 3;  
    while (n >= 0){  
        cout << n * n << endl;  
        --n;  
    }  
    cout << n << endl;  
    Return 0;  
}
```


Exercise – what is the output ?

```
int main() {  
    int n = 3;  
    while (n >= 0) {  
        cout << n * n << endl;  
        --n;  
    }  
    cout << n << endl;  
    Return 0;  
}
```

9
4
1
0
-1

Dummy test – Multiple Choice Questions

For each question

- 4 possible answers proposed
- one and only one answer is correct. Eg : 3.A (question 3, answer A)
- 1 point is awarded for each correct answer
- 0 points for no answer
- -0.2 points for each incorrect answer.

Dummy test – Multiple Choice Questions

Question #1 - C Language is :

- A) A compiled language
- B) An interpreted language
- C) Both compiled and interpreted
- D) A High Level Language

Dummy test – Multiple Choice Questions

Question #2 - Which function is called first when a program starts?

- A) main()
- B) class()
- C) loop()
- D) variable()

Dummy test – Multiple Choice Questions

Question #3 - Which of the following is NOT a characteristic of C language?

- A) Fast execution
- B) Platform independence through virtual machine
- C) Procedural programming
- D) Portability

Dummy test – Multiple Choice Questions

Question #4 - What is a variable in C language?

- A) A fixed value
- B) A keyword
- C) A preprocessor directive
- D) A memory location to store data

Dummy test – Multiple Choice Questions

Question #5 - What will be the value of x?

- A) 8
- B) 7.5
- C) 7
- D) Error

Dummy test – Multiple Choice Questions

Question #6 - What is a function in C?

- A) A block of code that performs a specific task.
- B) A keyword
- C) A type of variable
- D) A preprocessor directive

Dummy test – Multiple Choice Questions

Question #7 - How to declare a function in C that takes two float parameters and returns an integer?

- A) `int MyFunction(float, float);`
- B) `int MyFunction(a, b);`
- C) `int MyFunction( float a, float b);`
- D) `int MyFunction();`

Dummy test – Multiple Choice Questions

Question #8 - What is a pointer in C?

- A) A variable that stores the address of another variable
- B) A variable that stores data directly
- C) A keyword to create arrays
- D) None of the above

Dummy test – Multiple Choice Questions

Question #9 - Which symbol is used to declare a pointer?

A) &

B) *

C) #

D) %

Dummy test – Multiple Choice Questions

Question #10 - What is the output of the «&» operator?

- A) Value at address
- B) Value of variable
- C) Address of variable
- D) None

Dummy test – Multiple Choice Questions

Question #11 - What does the following pointer declaration mean?

```
int *p;
```

- A) p is an integer
- B) p is an address
- C) p is a pointer to an integer
- D) p is an function

Dummy test – Multiple Choice Questions

Question #12 - Which of the following correctly declares an array of 10 integers in C?

- A) `int arr(10);`
- B) `array int arr[10];`
- C) `int[10] arr;`
- D) `int arr[10];`

Dummy test – Multiple Choice Questions

Question #13 - What is the index of the first element in a C array?

- A) 0
- B) 1
- C) -1
- D) Depends on the Array.

Dummy test – Multiple Choice Questions

Question #14 - Which of the following is NOT a valid array declaration?

- A) `int arr[10];`
- B) `int arr[] = {1, 2, 3};`
- C) `int arr[3] = {1, 2, 3, 4};`
- D) `float marks[5];`

Dummy test – Multiple Choice Questions

Question #15 - What will be the output of this code?

```
int a = 7;
```

```
int b = 2;
```

```
int* ptr;
```

```
ptr = &a;
```

```
b = *ptr + 1;
```

```
printf("%d", b);
```

A) 9

B) 3

C) 7

D) 8

Multiple Choice Questions - Answers

Answers

Dummy test – Multiple Choice Questions

Question #1 - C Language is :

- A) A compiled language
- B) An interpreted language
- C) Both compiled and interpreted
- D) A High Level Language

Dummy test – Multiple Choice Questions

Question #1 - C Language is :

- A) A compiled language**
- B) An interpreted language
- C) Both compiled and interpreted
- D) A High Level Language

Dummy test – Multiple Choice Questions

Question #2 - Which function is called first when a program starts?

- A) main()
- B) class()
- C) loop()
- D) variable()

Dummy test – Multiple Choice Questions

Question #2 - Which function is called first when a program starts?

- A) **main()**
- B) class()
- C) loop()
- D) variable()

Dummy test – Multiple Choice Questions

Question #3 - Which of the following is NOT a characteristic of C language?

- A) Fast execution
- B) Platform independence through virtual machine
- C) Procedural programming
- D) Portability

Dummy test – Multiple Choice Questions

Question #3 - Which of the following is NOT a characteristic of C language?

- A) Fast execution
- B) Platform independence through virtual machine**
- C) Procedural programming
- D) Portability

Dummy test – Multiple Choice Questions

Question #4 - What is a variable in C language?

- A) A fixed value
- B) A keyword
- C) A preprocessor directive
- D) A memory location to store data

Dummy test – Multiple Choice Questions

Question #4 - What is a variable in C language?

- A) A fixed value
- B) A keyword
- C) A preprocessor directive
- D) **A memory location to store data**

Dummy test – Multiple Choice Questions

Question #5 - What will be the value of x?

- A) 8
- B) 7.5
- C) 7
- D) Error

Dummy test – Multiple Choice Questions

Question #5 - What will be the value of x?

A) 8

B) 7.5

C) 7

D) Error

Dummy test – Multiple Choice Questions

Question #6 - What is a function in C?

- A) A block of code that performs a specific task.
- B) A keyword
- C) A type of variable
- D) A preprocessor directive

Dummy test – Multiple Choice Questions

Question #6 - What is a function in C?

- A) A block of code that performs a specific task.**
- B) A keyword
- C) A type of variable
- D) A preprocessor directive

Dummy test – Multiple Choice Questions

Question #7 - How to declare a function in C that takes two float parameters and returns an integer?

- A) `int MyFunction(float, float);`
- B) `int MyFunction(a, b);`
- C) `int MyFunction( float a, float b);`
- D) `int MyFunction();`

Dummy test – Multiple Choice Questions

Question #7 - How to declare a function in C that takes two float parameters and returns an integer?

- A) `int MyFunction(float, float);`
- B) `int MyFunction(a, b);`
- C) **`int MyFunction( float a, float b);`**
- D) `int MyFunction();`

Dummy test – Multiple Choice Questions

Question #8 - What is a pointer in C?

- A) A variable that stores the address of another variable
- B) A variable that stores data directly
- C) A keyword to create arrays
- D) None of the above

Dummy test – Multiple Choice Questions

Question #8 - What is a pointer in C?

- A) A variable that stores the address of another variable**
- B) A variable that stores data directly
- C) A keyword to create arrays
- D) None of the above

Dummy test – Multiple Choice Questions

Question #9 - Which symbol is used to declare a pointer?

A) &

B) *

C) #

D) %

Dummy test – Multiple Choice Questions

Question #9 - Which symbol is used to declare a pointer?

A) &

B) *

C) #

D) %

Dummy test – Multiple Choice Questions

Question #10 - What is the output of the «&» operator?

- A) Value at address
- B) Value of variable
- C) Address of variable
- D) None

Dummy test – Multiple Choice Questions

Question #10 - What is the output of the «&» operator?

- A) Value at address
- B) Value of variable
- C) Address of variable**
- D) None

Dummy test – Multiple Choice Questions

Question #11 - What does the following pointer declaration mean?

```
int *p;
```

- A) p is an integer
- B) p is an address
- C) p is a pointer to an integer
- D) p is an function

Dummy test – Multiple Choice Questions

Question #11 - What does the following pointer declaration mean?

```
int *p;
```

- A) p is an integer
- B) p is an address
- C) p is a pointer to an integer**
- D) p is an function

Dummy test – Multiple Choice Questions

Question #12 - Which of the following correctly declares an array of 10 integers in C?

- A) `int arr(10);`
- B) `array int arr[10];`
- C) `int[10] arr;`
- D) `int arr[10];`

Dummy test – Multiple Choice Questions

Question #12 - Which of the following correctly declares an array of 10 integers in C?

- A) `int arr(10);`
- B) `array int arr[10];`
- C) `int[10] arr;`
- D) **`int arr[10];`**

Dummy test – Multiple Choice Questions

Question #13 - What is the index of the first element in a C array?

- A) 0
- B) 1
- C) -1
- D) Depends on the Array.

Dummy test – Multiple Choice Questions

Question #13 - What is the index of the first element in a C array?

- A) **0**
- B) 1
- C) -1
- D) Depends on the Array.

Dummy test – Multiple Choice Questions

Question #14 - Which of the following is NOT a valid array declaration?

- A) `int arr[10];`
- B) `int arr[] = {1, 2, 3};`
- C) `int arr[3] = {1, 2, 3, 4};`
- D) `float marks[5];`

Dummy test – Multiple Choice Questions

Question #14 - Which of the following is NOT a valid array declaration?

- A) `int arr[10];`
- B) `int arr[] = {1, 2, 3};`
- C) **`int arr[3] = {1, 2, 3, 4};`**
- D) `float marks[5];`

Dummy test – Multiple Choice Questions

Question #15 - What will be the output of this code?

```
int a = 7;
```

```
int b = 2;
```

```
int* ptr;
```

```
ptr = &a;
```

```
b = *ptr + 1;
```

```
printf("%d", b);
```

A) 9

B) 3

C) 7

D) 8

Dummy test – Multiple Choice Questions

Question #15 - What will be the output of this code?

```
int a = 7;
```

```
int b = 2;
```

```
int* ptr;
```

```
ptr = &a;
```

```
b = *ptr + 1;
```

```
printf("%d", b);
```

A) 9

B) 3

C) 7

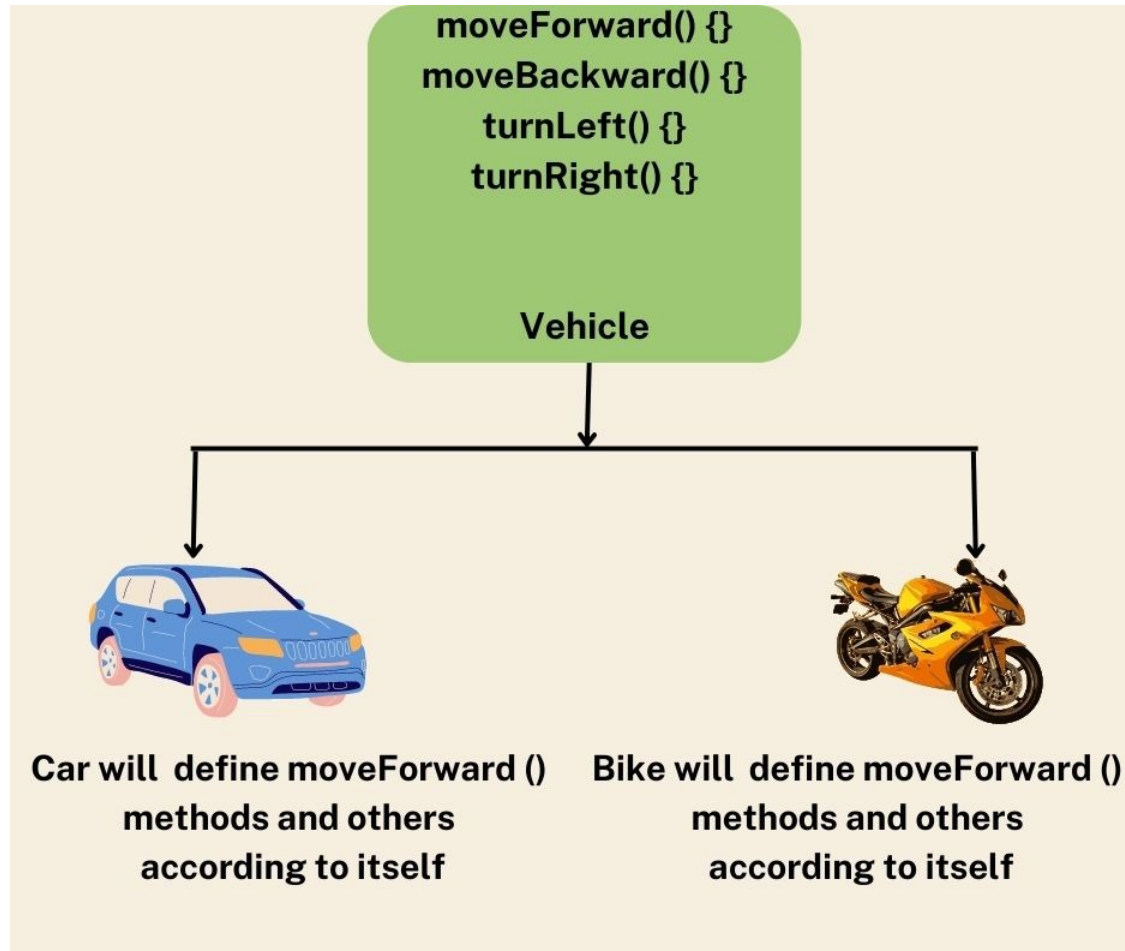
D) **8**

Multiple Choice Questions - Answers

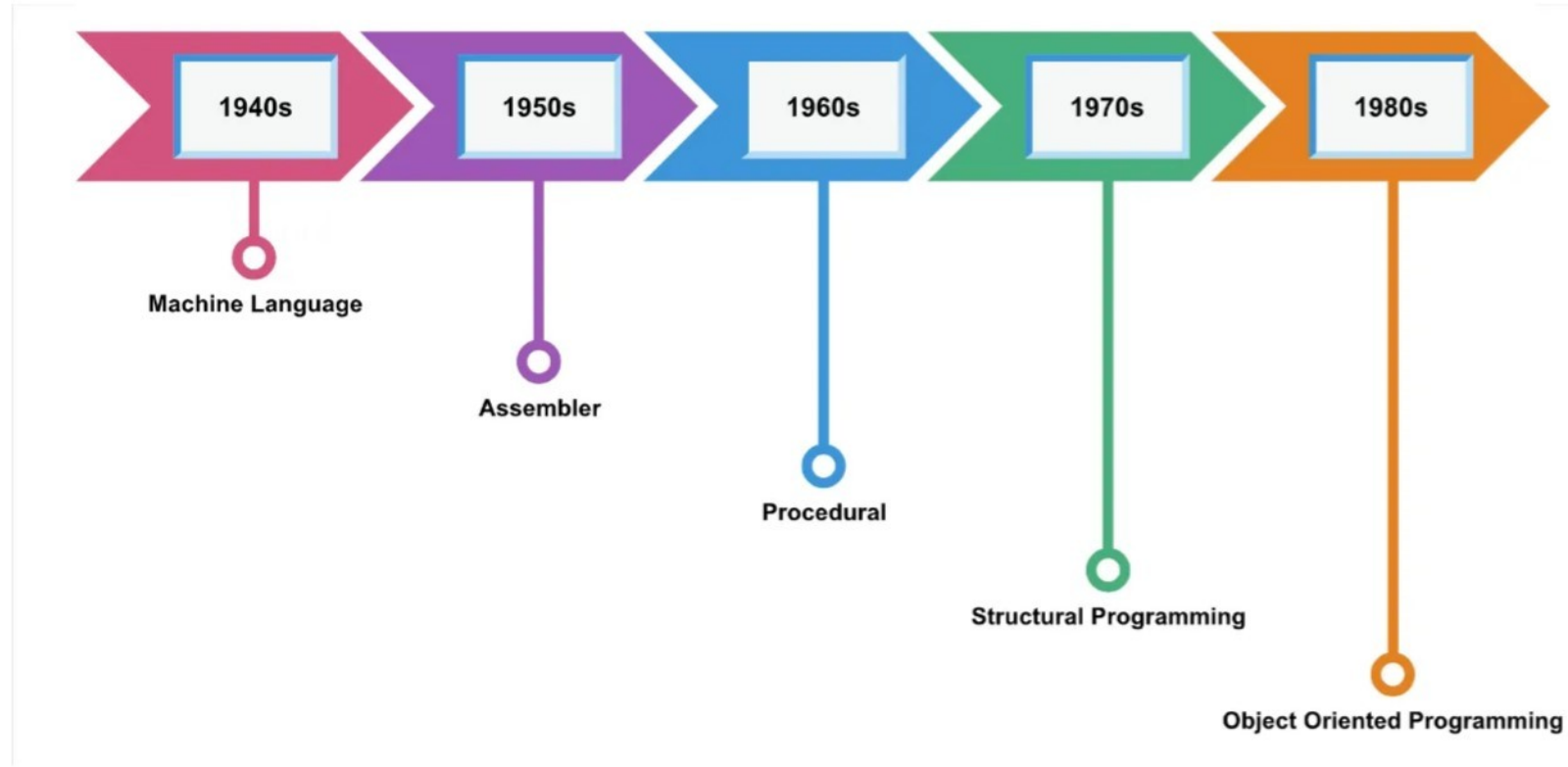
- 1A
- 2A
- 3B
- 4D
- 5C
- 6A
- 7C

- 8A
- 9B
- 10C
- 11C
- 12D
- 13A
- 14C
- 15D

Classes & Objects



Classes – OOP History at a glance



Structured programming dictated that a system be fully specified before a single line of code was written, leading to either manual documentation and

Classes – OOP History at a glance

- 1950s → 1970s : Early concepts & languages : LISP, MODULA...
- 1980's : C++
- Since : Java and other languages

Classes – OOP Key Motivators

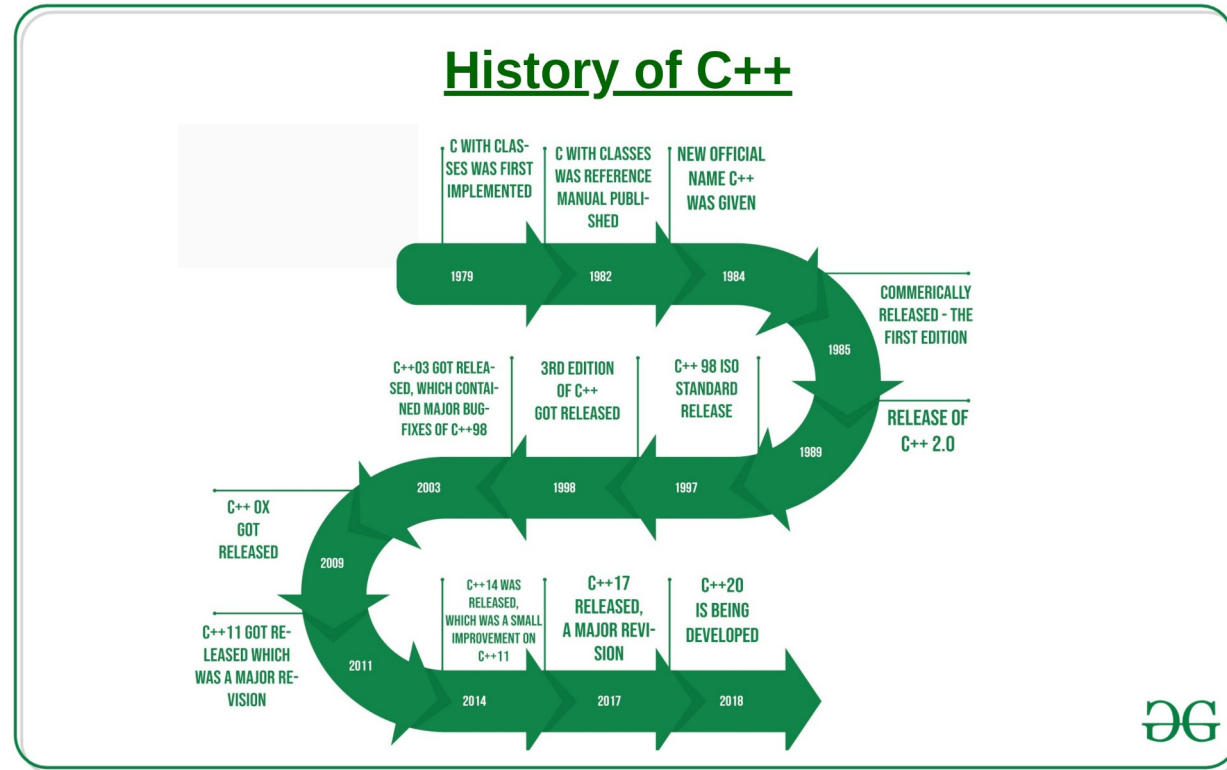
- quest for superior module structures for complex systems
- desire to devise more flexible versions of assignments.
- Supporting concepts of **abstraction** and **encapsulation**
- Supporting **inheritance** and **polymorphism**

C++ - « C with Classes »

1979 - « C with Classes »

1985 – C++

Since : evolutions



C++ - Object Oriented Programming

Class : a **user-defined data type** that we can use in our program, working as an **object constructor** (a "blueprint") for creating objects.

- Features **Attributes** and **methods : variables** and **functions** that belongs to the class. Referred to as "**class members**".

Object : a **particular instance** of a given **class**.

- A real life **car** is an **object**. It has attributes, such as « weight » and « color », and methods, such as « drive » and « brake ».

C++ - Object Oriented Programming

- Class : a cake recipe.
- Object : a cake made from that recipe.

Classes & Objects

Classes and objects are the two main aspects of object-oriented programming.

Class : Fruit

Objects : Apple, Banana, Mango

Class : Vehicle

Objects : Car, Bus, Bicycle, Truck

A class is a **template** for objects

An object is an **instance** of a class.

- When individual objects are created, they **inherit** all the variables and functions from the class.

C++ - Create a Class

```
class MyClass
{    // The class

public:    // Access specifier

int myNum;    // Attribute (int)

void myMethod(){
    cout << "Hello World!";
};

private:    // Access specifier

string myString; // Attribute (string)

};
```

The **class** keyword is used to create a class called **MyClass**.

class members (attributes and methods) under **public** keyword are accessible **from outside** the class.

class members (attributes and methods) under **private** are accessible **only from inside** the class.

the class definition ends with a « ; »

C++ - Create a Class

```
class MyClass {    // The class
public:            // Access specifier
int myNum;        // Attribute (int var)
};
```

```
int main() {
    MyClass myObj; // Create instance of
                    // MyClass
    // Access attributes and set values
    myObj.myNum = 15;
    cout << myObj.myNum << "\n";
    return 0;
}
```

C++ - Constructor

```
class MyClass {    // The class
```

```
public:
```

```
MyClass() {    // Constructor
```

```
    cout << "Hello World!";
```

```
};
```

```
~MyClass() {    // Destructor
```

```
    cout << "Bye Bye World!";
```

```
};
```

```
};
```

A constructor in C++ is a special method that is automatically called when an object of a class is created. It has the same name as the class, followed by parentheses ()

A destructor is a member function that is invoked automatically when the object goes out of scope or is explicitly destroyed by a call to delete or delete[]. It has the same name as the class preceded by a tilde (~).

Example – « Circle » Class

- Write a C++ **class Circle**
 - private member variable « **radius** ».
 - member functions (methods) calculating
 - circle's area
 - circle's circumference.
- Write a C++ main program using **Circle** that
 - Let user enter a radius
 - Display circle's area and circumference

Circle Class sample

```
#include <iostream>

#include <cmath>

const double PI = 3.14159; // Define PI as a constant

class Circle {
private:
    double radius; // Private member
public:
    Circle(double rad) : radius(rad) {} // Constructor
    double calculateArea() { // member function
        return PI * pow(radius, 2); // circle area
    }
    double calculateCircumference() { // Member function
        return 2 * PI * radius;
    }
};
```

Such Class code definition
can be hosted in a **circle.h**
header file

Circle Class usage

```
#include "circle.h"
int main() {
    double radius;
    std::cout << "Input the radius of the circle: ";
    std::cin >> radius; // Input the radius
    Circle circle1(radius); // Create Circle
    // Calculate and display circle area
    double area = circle1.calculateArea();
    std::cout << "Area: " << area << std::endl;
    // Calculate and display circle circumference
    double circumference = circle1.calculateCircumference();
    std::cout << "Circumference: " << circumference <<
std::endl;
    return 0;
}
```

Main code instantiates
and uses an object
circle1, inherited from
Circle Class

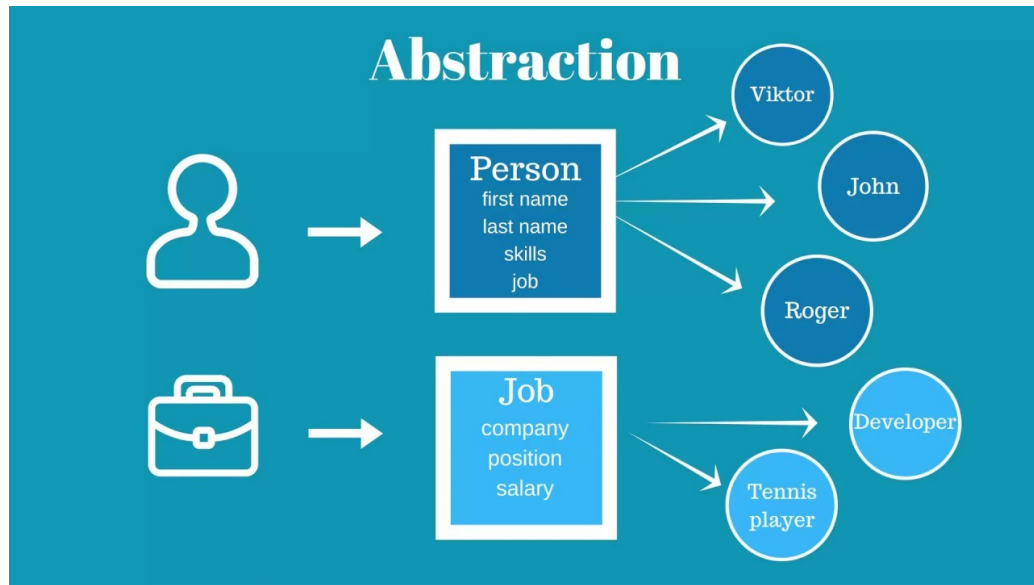
OOP Principles

- **Abstraction**
- **Encapsulation**
- Inheritance
- Polymorphism

Abstraction

Abstraction helps you focus on a system's essential elements and ignore the less important details that have no effect on its key features. It allows you to construct more understandable programs. Abstraction can be thought of as an expansion of encapsulation. Take programs that contain thousands of lines of code as an example. Through the principle of abstraction, each object only reveals a specific mechanism for usage. Therefore, the code inside becomes largely independent of other objects.

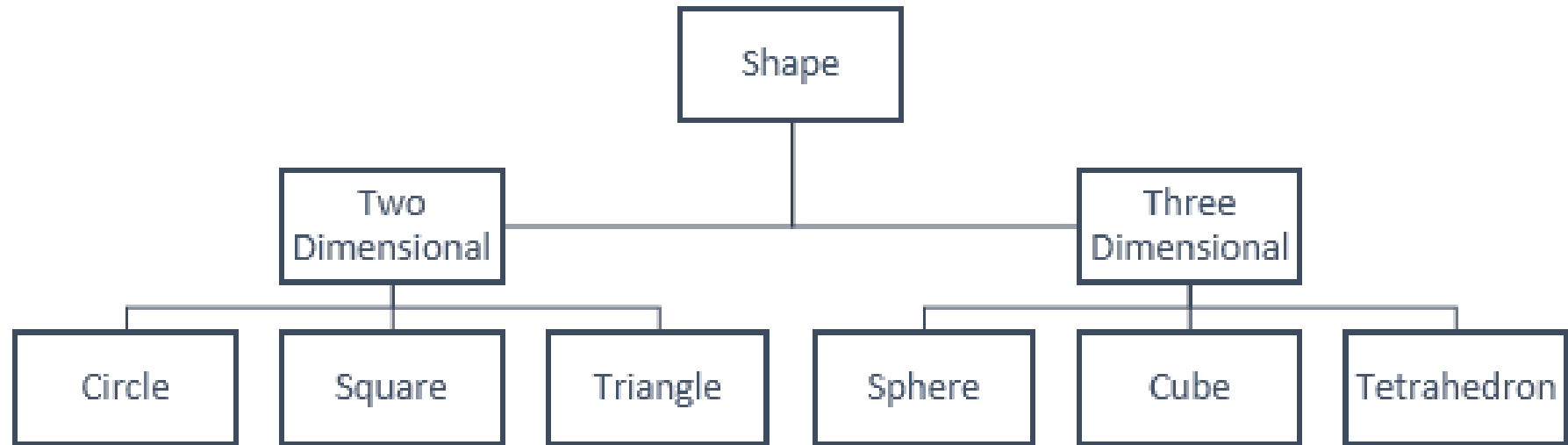
- For instance, in a program that stores information about movies, you can create a class "Movie" that provides access to only the most essential details, such as title, release year, and genre, while hiding the less important information, like shots or technical aspects.



Abstraction : Define and use Classes

- One of the most essential features of OOP in C++.
- Means : display only essential information – ignore details.
- Example of a person driving a car :
 - Driver knows : pressing the accelerator increases the speed
 - Does NOT know : **how** speed is actually increased.

Abstraction in C++



Encapsulation

Encapsulation seeks to hide the implementation details of objects from the outside world. It states that all important information is contained within the object; only selected data is available externally. Each object's inner workings and state are stored privately within the specified class, whereas other objects do not have access to it or the ability to make changes. Instead, they can only interact with a few public functions or methods. This form of data hiding provides program security and control over object state changes, reduces the risk of errors, and makes the program more understandable.

Encapsulation in C++



Abstraction vs Encapsulation

- **Abstraction** reduces complexity by focusing on the essential features needed for a particular context, without delving into the underlying technicalities.
- **Encapsulation** secures data and functions within a class, preventing unauthorized access and modification.

OOP Principles

- Abstraction
- Encapsulation
- **Inheritance**
- **Polymorphism**

Problem - define new similar Classes

Write C++ **classes**
Rectangle, **Triangle**,
Square featuring
methods calculating
area and perimeter.

- How to proceed?

Problem - define new Classes

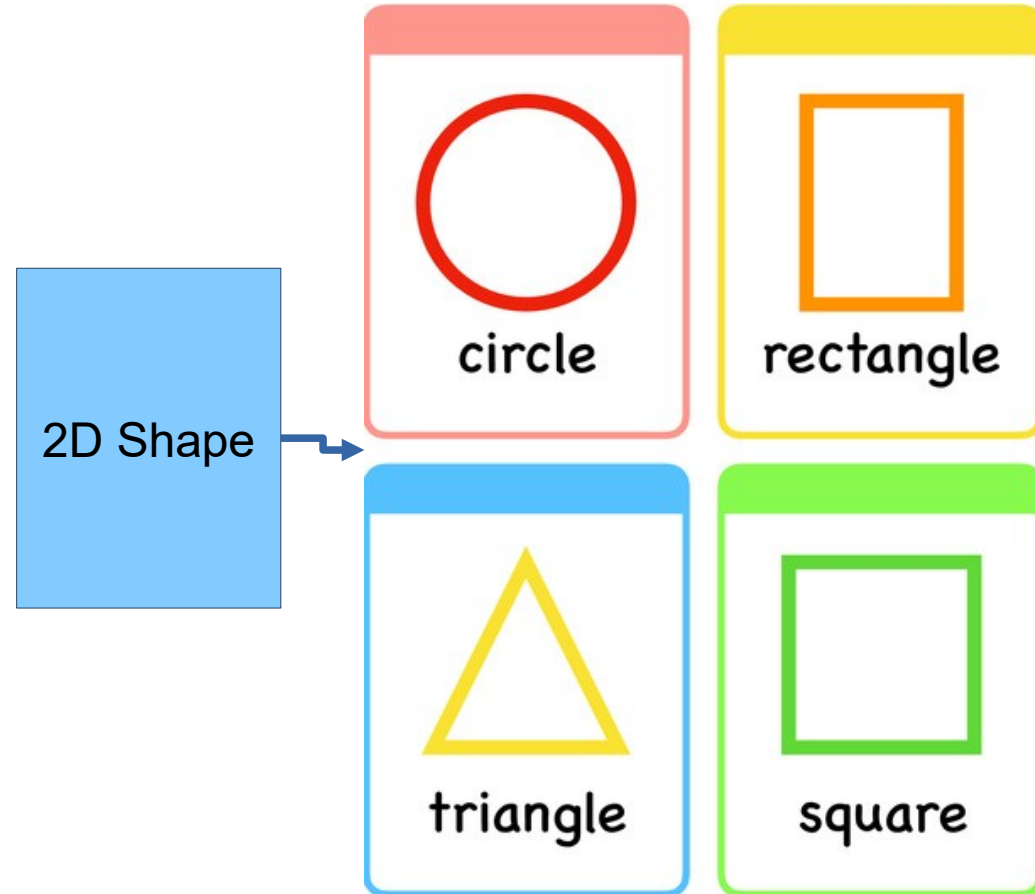
Either

« cut and paste » circle's
class code and redefine
what's different, i.e. area
and perimeter (instead
of circumference)
formulae/methods for
each new class

- OR
- refactor classes
by abstraction

Refactoring by Abstraction

- Identify common functionality shared by multiple classes or methods and extract it into a separate, **abstract class or interface**. This process helps reduce code duplication, promotes reusability, and makes it easier to manage and maintain the shared functionality.
- For example, consider two classes with similar methods that perform same calculations. By extracting the shared logic into an abstract class, you can create a single implementation that both classes can inherit or implement. This reduces the amount of code and makes it easier to update or modify in the future.



Exercise – Shape Class & Inheritance

- Write a C++ program to implement a class called Shape2D with virtual member functions for calculating area and perimeter.
- Derive classes **Circle**, **Rectangle**, and **Triangle** from the Shape2D class and override virtual functions accordingly.

2D Shape Class

```
class Shape2D { // Define a base class named Shape
public:
    // Virtual member function to calculate area
    virtual double calculateArea() const = 0;
    // Virtual member function calculating perimeter
    virtual double calculatePerimeter() const = 0;
};
```

Circle derived from Shape2D

```
class Circle : public Shape2D { // Derived class Circle inheriting from Shape
private:
    double radius; // Private member variable : radius of the circle
    // Constructor for Circle class
    Circle(double rad) : radius(rad) {}
    // Override the virtual member function to calculate the area
    double calculateArea() const override {
        return PI * pow(radius, 2); // Calculate circle area
    }
    // Override the virtual member function to calculate perimeter
    double calculatePerimeter() const override {
        return 2 * PI * radius; // Calculate perimeter of the circle
    }
};
```

Square

```
#include "Shape2D.h"
class Square : public Shape2D { // Derived class inheriting from Shape2D
private:
    double sidelength; // Private member variable to store the side length of the square
public:
    Square(double sidelen) : sidelength(sidelen) {} // Constructor for Square class
    // Override the virtual member function to calculate the area
    double calculateArea() const override {
        return sidelength * sidelength; // area of the square
    }
    // Override the virtual member function to calculate the perimeter
    double calculatePerimeter() const override {
        return 4 * sidelength; // perimeter of a square
    }
};
```

Rectangle

```
#include "Shape2D.h"
class Rectangle : public Shape2D { // Define a derived class Rectangle inheriting from Shape
private:
    double length; // Private member variable : length of the rectangle
    double width; // Private member variable : width of the rectangle
public:
    // Constructor for Rectangle class
    Rectangle(double len, double wid) : length(len), width(wid) {}
    // Override the virtual member function to calculate the area
    double calculateArea() const override {
        return length * width;
    }
    // Override the virtual member function to calculate the perimeter
    double calculatePerimeter() const override {
        return 2 * (length + width);
    }
};
```

Triangle

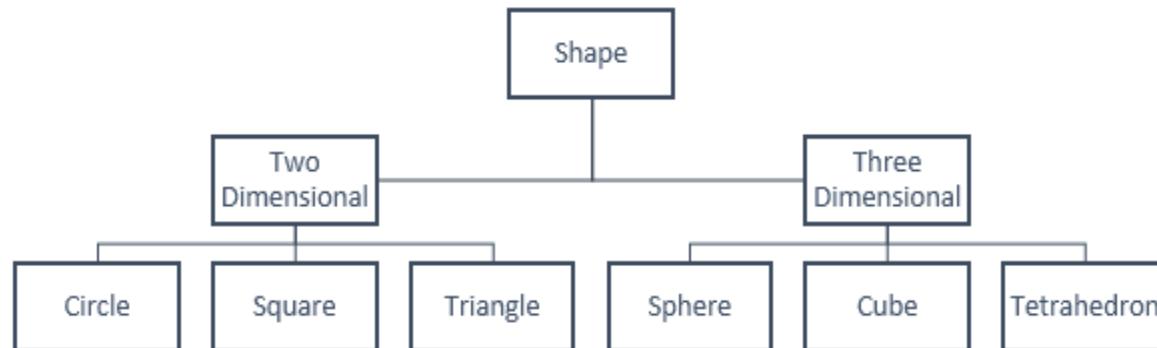
```
#include "Shape2D.h"
class Triangle : public Shape2D { // Define a derived class named Triangle inheriting from Shape
private:
    double side1; // Private member variable - first side of the triangle
    double side2; // Private member variable - second side of the triangle
    double side3; // Private member variable - third side of the triangle
public:
    // Constructor for Triangle class
    Triangle(double s1, double s2, double s3) : side1(s1), side2(s2), side3(s3) {}
    // Override the virtual member function to calculate the area
    double calculateArea() const override {
        // Using Heron's formula to calculate the area of a triangle
        double s = (side1 + side2 + side3) / 2; // semi-perimeter of the triangle
        return sqrt(s * (s - side1) * (s - side2) * (s - side3)); // area using Heron's formula
    }
    // Override the virtual member function to calculate the perimeter
    double calculatePerimeter() const override {
        return side1 + side2 + side3; // perimeter of the triangle using its sides
    }
};
```

Shape2D Classes usage

```
int main() {  
    // Create instances of different shapes: Circle, Rectangle, and Triangle  
    Circle circle(7.0); // Create a Circle object with radius 7.0  
    Rectangle rectangle(4.2, 8.0); // Create a Rectangle object with length 4.2 and width 8.0  
    Triangle triangle(4.0, 4.0, 3.2); // Create a Triangle object with sides 4.0, 4.0, and 3.2  
    // Calculate and display the area and perimeter of each shape  
    std::cout << "Circle: " << std::endl;  
    std::cout << "Area: " << circle.calculateArea() << std::endl; // Calculate and output the area of the circle  
    std::cout << "Perimeter: " << circle.calculatePerimeter() << std::endl; // Calculate and output the perimeter of the circle  
    std::cout << "Rectangle: " << std::endl;  
    std::cout << "Area: " << rectangle.calculateArea() << std::endl; // Calculate and output the area of the rectangle  
    std::cout << "Perimeter: " << rectangle.calculatePerimeter() << std::endl; // Calculate and output the perimeter of the  
rectangle  
    std::cout << "Triangle: " << std::endl;  
    std::cout << "Area: " << triangle.calculateArea() << std::endl; // Calculate and output the area of the triangle  
    std::cout << "Perimeter: " << triangle.calculatePerimeter() << std::endl; // Calculate and output the perimeter of the  
triangle  
    return 0; // Return 0 to indicate successful completion  
}
```


Inheritance

Inheritance is a principle that enables developers to create new classes based on existing (parent) classes, with the option of overriding or augmenting their properties and methods. This is especially useful in programs containing thousands of lines of code, as it simplifies maintenance and prevents code duplication. By using the logic of the parent class in the child class, developers can create objects that share code or logic yet are different. This reduces the complexity of the code and eliminates the need to create a new object for each object used in the program.



Inheritance

```
class Animal {  
public:  
    void eat() {  
        std::cout << "Animal is eating." << std::endl;  
    }  
};  
  
class Dog : public Animal {  
public:  
    void bark() {  
        std::cout << "Dog is barking." << std::endl;  
    }  
};
```

```
int main() {  
    Dog dog;  
    dog.eat(); // Animal is eating.  
    dog.bark(); // Dog is barking.  
    return 0;  
}
```

Polymorphism

Polymorphism is a principle that complements inheritance by allowing objects of different classes to **perform actions with the same name using different code**. For instance, the "show information" method can be used to display varied data about objects of the "car", "plane", or "ship" type. Furthermore, polymorphism helps to create more flexible and modular programs. Generally, it simplifies the development process as it allows for the creation of common methods and functions to be used for multiple types of objects.

Polymorphism

SAME NAME, DIFFERENT CODE

// Circle Class

```
double calculatePerimeter() const override {  
    return 2 * PI * radius; // perimeter of a circle
```

// Square Class

```
double calculatePerimeter() const override {  
    return 4 * sidelength; // perimeter of a square
```

Coffee

Hey, what's your name?



Right, what a stupid question.
I apologize, silly me.
I recognize the logo now.



Sorry, bye!
wow, how cold!



Break



...Anyway, I'm C++. So, er.. what's your best quality? For me I'd say it's speed.



But I can be a bit verbose too, all this templating these days, am I right? haha.



Python!



Exercise : write and use a Student Class

- Data members:
 - name(char type), marks1,marks2 (integers)
- Methods
 - **Constructor** (arguments = name, marks1, marks2)
 - **calc_media()** (Private) do the averaging of marks1 and marks2
 - **Disp** (public) displaying info to the output console
- Write main code : ask user to enter name and 2 marks. Instantiate a **Student** Class ; use it to display name and total media mark .

Exercise : write and use a Student Class

```
class Student {  
public:  
    char* name;  
    int mark1; int mark2;  
    Student(char* na, int ma1, int ma2) {  
        name = na; mark1 = ma1; mark2 = ma2;  
    }  
    void disp() {  
        cout << "Student:" << name << " \n media:" << calc_media() << "\n";  
    }  
private:  
    int calc_media() { return (mark1 + mark2) / 2; }  
};
```

```
int main() {  
    char nam[50]; int m1, m2;  
    cout << "Enter name:";  
    cin >> nam;  
    cout << "Enter marks of two subjects:";  
    cin >> m1;  
    cin >> m2;  
    Student student1(nam, m1, m2);  
    student1.disp();  
    return 0;  
}
```

Exercise : write and use a Car Class

Create a **Car** class with attributes **brand**, **model**, and **year**. Provide methods to set and display these attributes.

- **Data members:**

`string brand;`

`string model;`

`int year;`

- **Methods**

constructor

`void setDetails(string b, string m, int y) { [...]}`

`void displayDetails() {[...]}`

Exercise : write and use a Car Class

```
class Car {  
private:  
    string brand;  
    string model;  
    int year;  
public:  
    // Constructor  
    Car(string b, string m, int y) : brand(b), model(m), year(y) {}  
    // Setter method  
    void setDetails(string b, string m, int y) {  
        brand = b;  
        model = m;  
        year = y; }  
    // Display method  
    void displayDetails() {  
        cout << "Brand: " << brand << ", Model: " << model << ", Year: " << year << endl; }  
};
```

```
int main() {  
    Car myCar("Toyota", "Camry",  
2020);  
    myCar.displayDetails();  
    // Update details  
    myCar.setDetails("Honda", "Civic",  
2022);  
    myCar.displayDetails();  
    return 0;  
}
```

Evaluation - Project Selection

- Create Groups of 1, 2 or 3 (max) participants
- Each group to develop a C/C++ based program by improving one of the proposed projects
- OR
- develop one of your own once agreed on scope

Evaluation - Project Goal

- project based on an existing code :
 - Take over control on the code : read it, run it, understand it
 - improve it : « Hardening », Adding new features, Improving comments & structure, and **document it**
 - Send me your work by March 16th (1 day before presentation)
 - Present your work on March 17th

Evaluation - Project Goal

- If you choose a project of your own
 - Present it to me this upcoming February 23th
 - Choose something doable
 - Once agreed same rules apply
 - Implement features, make it work
 - make it « decently » reliable, attractive and useful
 - Code of decent quality : comments, structure, documentation

Project expected Deliverables

- Visual Studio (or equivalent) updated Project (**source code**)
- **Slides** supporting presentation
- Actual Presentation **including a demo** on October 24th

Evaluation - Project guidelines

- Select a project at your level, doable within a few weeks of good work
- Work on it from day #1 - do not wait last days
- Ask for support **early** and **often** (« derisking »)
- In slides, **explain your project development**, not only the final product
- Expose improved and new features, but also **difficulties**, design **choices**, **features chosen, dropped and why**, future work that could be done, etc.
- For groups (2 and 3), **explain everyone's role in the project**

Evaluation – Criteria

- Attendance / participation
- presentation & design
- features (added / improved)
- code quality
- code complexity and difficulty